MindCraft

Documentation technique - version publique

Architecture, infrastructure, conformité et modèle de données

Version du document	1.3 - version publique (allégée)
Date	7 novembre 2026
Version de MindCraft décrite	0.2.0 et évolutions de la branche `main` à cette date
Plateforme	https://www.mindcraft-research.fr
Dépôt	https://github.com/mindcraft-research/mindcraft
Licence	GNU AGPL-3.0-or-later
DOI	v0.2.0 : 10.5281/zenodo.23466120 - v0.1.0 : 10.5281/zenodo.19864887
SWHID (v0.1.0)	swh:1:rev:df8a8b127f4b7c024a140e56263d0e22927b33eb
ORCID de l'auteure	0000-0002-4315-1058 (Dayle DAVID)
Contact	contact@mindcraft-research.fr

À propos de ce document

Ce document décrit le fonctionnement technique de MindCraft, plateforme libre et sans code pour concevoir des études en ligne et en collecter les données. Il s'adresse aux équipes informatiques (DSI, ESI), aux délégué·e·s à la protection des données et aux personnes qui veulent contribuer au code.

> **Vous n'êtes pas informaticien·ne ?** Chaque section commence par un encadré « En clair ». Les mots techniques sont expliqués dans le glossaire (annexe C). Pour apprendre à utiliser MindCraft, consultez plutôt le guide utilisateur : menu Documentation du site.

Portée de cette version publique

Ce document est une version allégée destinée à la publication ouverte. Les éléments opérationnels qui exposeraient inutilement la plateforme sont résumés sans valeurs précises : seuils de limitation de débit et d'alerte, configuration interne de l'authentification, secrets de déploiement, objectifs de reprise chiffrés.

La version intégrale est disponible sur demande motivée à contact@mindcraft-research.fr pour les DPO, les équipes DSI/ESI dans le cadre d'une évaluation institutionnelle, les rapporteur·e·s scientifiques et les auditeur·rice·s sécurité mandaté·e·s.

Sommaire

1. Présentation générale
2. Composants techniques
3. Architecture applicative
4. Infrastructure et hébergement
5. Sécurité (résumé)
6. Intégration et déploiement continus
7. Conformité RGPD
8. Performance et montée en charge
9. Modèle de données
10. Supervision
11. Open source et communauté
12. Moteur expérimental et mesures physiologiques
13. Contrebalancement et allocation
Annexe A. Routes de l'API
Annexe B. Variables d'environnement (résumé)
Annexe C. Glossaire

1. Présentation générale

En clair. MindCraft est un site web qui permet de créer une étude sans programmer, de la faire passer à des participants et de récupérer les données. Il est gratuit, son code est public et il est hébergé en France.

MindCraft s'adresse aux chercheur·e·s, étudiant·e·s et praticien·ne·s des sciences humaines et comportementales. Il réunit une interface sans code, des tâches comportementales avec mesure du temps de réaction, le contrebalancement automatique, un hébergement français et une licence libre.

1.1 Principes de conception

- **Sans code :** une étude se construit par une interface graphique et s'enregistre en base de données.
- **Protection des données dès la conception :** hébergement en France, participants identifiés par un identifiant aléatoire, consentement intégré.
- **Science ouverte :** préenregistrement, cahier de laboratoire, codebook, préparation du dépôt, export de la structure de l'étude, plateforme citable.
- **Réversibilité :** données exportables à tout moment, code publié sous AGPL pour permettre l'auto-hébergement.
- **Reproductibilité :** copie de l'étude conservée au premier participant, ordre et conditions enregistrés pour chaque passation.

1.2 Ce que MindCraft n'est pas

- **Un outil d'analyse statistique :** les exports sont destinés à R, Python, JASP, SPSS ou jamovi.
- **Un panel de participants :** MindCraft se combine avec Prolific ou d'autres plateformes, mais n'en fournit pas.
- **Un système d'acquisition physiologique :** MindCraft émet des marqueurs LSL, mais n'enregistre pas le signal des capteurs.
- **Un service commercial :** l'instance hébergée sur www.mindcraft-research.fr est réservée à la recherche et à l'enseignement.
- **Une solution clé en main pour les données sensibles** (santé, biométrie au sens de l'article 9 du RGPD) : ces usages nécessitent une analyse d'impact spécifique.

2. Composants techniques

En clair. La liste des briques logicielles utilisées. Toutes sont libres ou d'usage courant, et leurs versions sont fixées dans le dépôt.

Rôle	Technologie	Version
Interface	Next.js (Pages Router) et React	14.2 / 18
État côté navigateur	Zustand	4
Échanges de données	TanStack Query	5
Serveur (API)	Fastify sur Node.js	4 / Node 20
Accès à la base	Prisma	5
Base de données	PostgreSQL	16
Authentification	JWT, bcrypt, TOTP (otplib)	-
E-mails transactionnels	Resend	-
PDF (codebook, préenregistrement, cahier)	PDFKit	-
Tableurs	ExcelJS, SheetJS (xlsx, ODS)	-
Images	Sharp	-
Nettoyage du HTML	sanitize-html, DOMPurify	-
Conteneurs	Docker (construction en plusieurs étapes)	-
Intégration continue	GitHub Actions	-
Tests	Vitest	4

L'application est écrite en JavaScript, côté navigateur comme côté serveur. PostgreSQL combine des tables relationnelles (comptes, projets, études) et des champs JSON pour les réglages propres à chaque type de bloc ou de question.

3. Architecture applicative

En clair. Trois parties travaillent ensemble : le site que l'on voit, un serveur qui applique les règles, et une base de données qui conserve tout.

3.1 Trois niveaux

Niveau	Rôle	Adresse
Interface (Next.js)	Pages affichées dans le navigateur	www.mindcraft-research.fr
API (Fastify)	Échanges REST en JSON, règles d'accès	api.mindcraft-research.fr
Base (PostgreSQL)	Conservation des données	Accès réservé à l'API

3.2 Interface

Le code se trouve dans `frontend/src/` : `pages/` (routes), `components/` (dont `builder/` pour l'éditeur d'études, `runner/` pour la passation et `stimulus/` pour les tâches), `lib/` (services partagés) et `middleware.js` (passage en HTTPS). Les pages sont pré-générées par Next.js, puis chargent leurs données depuis l'API une fois dans le navigateur.

Pendant une passation, l'interface reçoit la structure de l'étude et déroule le parcours dans le navigateur du participant : c'est ce qui limite la latence pendant les tâches.

3.3 Serveur

Le serveur, dans `backend/src/`, enregistre ses modules dans `server.js` : CORS, cookies, JWT, limitation de débit, en-têtes de sécurité et envoi de fichiers. Les routes sont réparties par domaine dans `routes/` :

Fichier	Domaine
`auth.js`	Comptes, connexion, double authentification
`projects.js`, `studies.js`, `design.js`	Projets, études, blocs, questions, design, sessions
`run.js`	Passation publique
`stimulus.js`, `media.js`	Tâches et fichiers
`export.js`, `anonymisation.js`	Exports et masquage des réponses libres
`prereg.js`, `notebook.js`	Préenregistrement et cahier de laboratoire
`admin.js`, `feedback.js`	Administration et retours des utilisateurs

3.4 Communication

L'interface et le serveur échangent en HTTP et JSON, sans GraphQL. Seule exception : pour les mesures physiologiques, le navigateur envoie ses marqueurs en WebSocket à un petit programme local, le relais LSL (section 12).

4. Infrastructure et hébergement

En clair. Le site et les données sont hébergés en France, chez Scaleway.

4.1 Composants

Composant	Service	Rôle
Interface et API	Scaleway Serverless Containers	Exécution des conteneurs, HTTPS, montée en charge automatique
Base de données	Scaleway Managed PostgreSQL	PostgreSQL 16, sauvegardes automatiques
Images Docker	Scaleway Container Registry	Registre privé (région fr-par)
Nom de domaine	OVH	DNS

Les e-mails transactionnels (vérification d'adresse, réinitialisation, invitations) sont envoyés par Resend.

4.2 Secrets et configuration

Aucun secret n'est stocké dans le code source. Trois zones coexistent : développement local (fichier `.env` ignoré par Git), intégration continue (secrets chiffrés de GitHub) et production (variables d'environnement chez Scaleway). Voir l'annexe B.

4.3 Sauvegardes

Le code source est conservé sur GitHub et archivé par Software Heritage et Zenodo pour les versions publiées. Les images Docker sont conservées dans le registre, et la base bénéficie des sauvegardes automatiques de Scaleway. Les objectifs de reprise chiffrés figurent dans la version intégrale.

5. Sécurité (résumé public)

En clair. Les mots de passe ne sont pas conservés en clair, les échanges sont chiffrés, et chaque action de l'espace chercheur vérifie vos droits. Les valeurs précises des réglages ne sont pas publiées ici.

5.1 Authentification

- Mots de passe hachés avec bcrypt, longueur minimale vérifiée par le serveur.
- Adresse e-mail à vérifier avant la première connexion.
- Jetons JWT signés : un jeton d'accès court et un jeton de rafraîchissement dans un cookie HttpOnly, Secure et SameSite=Strict.
- Se déconnecter ou changer de mot de passe invalide les jetons des autres appareils.
- Double authentification facultative par code temporaire (TOTP, RFC 6238).
- Liens de vérification et de réinitialisation expirants, conservés sous forme d'empreinte.
- Changement d'adresse e-mail soumis au mot de passe et à une confirmation par lien.

5.2 Autorisation

Dans l'espace chercheur, chaque route vérifie le jeton, puis le rôle de la personne dans le projet (`lib/acces.js`) : propriétaire, éditeur ou lecteur. Un rôle d'administrateur de la plateforme s'y ajoute.

La passation se fait sans compte : ses routes sont publiques par conception. Pour enregistrer des données, le serveur vérifie que la session existe et que le bloc appartient à l'étude.

5.3 Protections

Risque	Protection
Injection de script (XSS)	sanitize-html côté serveur, DOMPurify côté navigateur
Requêtes forgées (CSRF)	Cookies SameSite=Strict, liste d'origines CORS autorisées
Injection SQL	Requêtes paramétrées par Prisma
Force brute	Limitation de débit (`@fastify/rate-limit`)
En-têtes HTTP	`@fastify/helmet`, dont HSTS
Connexions non chiffrées	Redirection vers HTTPS (middleware Next.js)
Accès aux fichiers	Chemins contrôlés côté serveur
Fichiers envoyés	Liste de types autorisés, taille limitée

5.4 Traçabilité

Les actions sur les projets et les études sont journalisées dans la table `ActivityLog`. Pour chaque étude, le cahier de laboratoire conserve une trace datée et non modifiable des événements importants.

5.5 Signaler une faille

Voir `SECURITY.md` : merci d'utiliser une voie confidentielle (contact@mindcraft-research.fr, ou l'avis de sécurité privé de GitHub) et de ne pas ouvrir d'issue publique.

6. Intégration et déploiement continus

En clair. Chaque modification du code passe par des tests automatiques. Une fois intégrée à la branche principale, elle est mise en ligne automatiquement.

Le fichier `.github/workflows/ci.yml` s'exécute à chaque proposition de modification et à chaque envoi sur `main` :

- **backend-test :** installation, création du schéma sur une base PostgreSQL de test, démarrage du serveur, tests Vitest.
- **frontend-build :** construction du site.
- **build-images** (sur `main`) : construction des images Docker et envoi au registre Scaleway.
- **deploy** (sur `main`) : redéploiement des conteneurs.

6.1 Schéma de la base

Le schéma est décrit dans `backend/prisma/schema.prisma`. Il est appliqué par `prisma db push` au démarrage du serveur, sans fichiers de migration versionnés.

6.2 Conteneurs

Les Dockerfiles construisent l'application en plusieurs étapes pour alléger l'image finale, sur une base Node 20 (Debian slim pour le serveur, Alpine pour l'interface).

6.3 Protection de la branche et dépendances

La branche `main` est protégée par une règle GitHub. Dependabot propose chaque semaine des mises à jour des dépendances npm, des actions GitHub et des images Docker.

6.4 Versions

Les versions suivent une numérotation `v0.x.y`, phase actuelle. Une version publiée est archivée sur Zenodo, avec son DOI.

7. Conformité RGPD

En clair. MindCraft enregistre le minimum sur les participants : pas de nom ni d'adresse e-mail par défaut. Le ou la responsable de l'étude reste responsable de ce qu'il ou elle demande aux participants, y compris dans les réponses libres.

7.1 Correspondance avec le règlement

Exigence	Mise en œuvre	Article
Localisation	Hébergement en France (Scaleway)	Chap. V
Minimisation	Type d'appareil seulement, pas le texte complet d'identification du navigateur	5.1.c
Limitation des finalités	Recherche et enseignement, pas de marketing	5.1.b
Chiffrement en transit	HTTPS	32
Pseudonymisation	Identifiant aléatoire (UUID), ou identifiant Prolific le cas échéant	25
Protection des mots de passe	Hachage bcrypt	32
Droit d'accès et portabilité	Chercheur·e : export de ses données. Étude : exports CSV, Excel, ODS	15, 20
Droit à l'effacement	Suppression du compte ; effacement des données d'un seul participant ou de toute une étude	17
Droit de rectification	Modification du profil	16
Consentement	Question Consentement avec boutons Accepter / Refuser	7
Anonymisation avant partage	Masquage des réponses libres, identifiants remplacés dans le paquet de dépôt	25, 89
Responsabilité	Journal d'activité, cahier de laboratoire	5.2

Les conditions d'utilisation (section 5) contiennent l'accord de sous-traitance prévu par l'article 28, à transmettre au DPO de l'établissement.

7.2 Sous-traitants et services

Service	Rôle	Données concernées
Scaleway	Hébergement (conteneurs, base, registre)	Toutes
OVH	Nom de domaine	Aucune donnée de recherche
Resend	E-mails transactionnels	Adresses des chercheur·e·s, et des auteurs à qui un préenregistrement est envoyé sur demande
GitHub, Zenodo, Software Heritage	Code source public et archives	Code uniquement

7.3 Données collectées

Chercheur·e·s

Nom d'utilisateur, adresse e-mail vérifiée, mot de passe haché, secret de double authentification s'il est activé, profil facultatif, journal d'activité.

Participants

- Identifiant aléatoire, ou identifiant Prolific s'il est transmis par le lien.
- Réponses, temps de réponse, essais (touche, exactitude, temps de réaction), résultats et marqueurs des tâches externes.
- Dates de passation, heure d'arrivée sur chaque page, condition et ordre des blocs, langue, type d'appareil, version de l'étude.
- Aucune table ne prévoit de nom, d'adresse e-mail ou d'adresse IP de participant. Un chercheur peut toutefois en demander via ses questions, et un participant peut en écrire dans une réponse libre.

7.4 Droits des personnes

Chercheur·e·s : export et suppression du compte depuis les paramètres, modification du profil.

Participants : la plateforme ne connaît pas leur identité. Ils s'adressent au ou à la responsable de l'étude, qui peut effacer les données d'une personne à partir de son identifiant (« Réinitialiser les données » puis « Un seul participant »).

8. Performance et montée en charge

En clair. La plateforme supporte l'arrivée simultanée de nombreux participants, par exemple en amphithéâtre : ils sont traités l'un après l'autre, sans message d'erreur.

- **Conteneurs sans état :** aucune session locale, chaque requête peut être traitée par n'importe quelle instance.
- **Arrivées simultanées :** une file d'attente par étude, et un verrou en base (`pg_advisory_xact_lock`) pour le cas où plusieurs instances tournent en parallèle.
- **Cache :** TanStack Query côté navigateur. Pas de cache externe côté serveur.
- **Index :** déclarés dans le schéma Prisma sur les champs les plus consultés.

> Les paramètres de montée en charge sont documentés dans la version intégrale.

9. Modèle de données

En clair. Un compte possède des projets, un projet contient des études, une étude contient des blocs, un bloc contient des questions. Les réponses des participants sont rangées à part.

Le schéma complet est public : `backend/prisma/schema.prisma`.

9.1 Principales tables

Table	Contenu
`User`	Compte : authentification, profil, rôle, double authentification
`Project`, `Collaborator`, `Invitation`	Projet, membres (éditeur, lecteur) et invitations
`Study`	Étude. Statut : DRAFT, REVIEW, VALIDATED, COLLECTING, ARCHIVED
`StudyVersion`, `StudyFreeze`, `StudyPreviewKey`	Versions, copie de l'étude au premier participant, clé du lien d'aperçu
`Block`	Bloc : WELCOME, INSTRUCTION, QUESTION, STIMULUS, LOGIC, DEBRIEFING
`Question`, `Choice`, `MatrixItem`, `Condition`	Questions, choix, lignes de matrice, conditions d'affichage
`StimulusFile`, `TrialSequenceStep`, `MediaFile`	Fichiers des tâches, étapes d'un essai, médias
`ExperimentalDesign`, `Factor`, `FactorLevel`	Design, facteurs et niveaux
`ParticipantSession`, `ConditionAssignment`, `RandomCode`	Passation, condition attribuée, code aléatoire
`QuestionResponse`, `TrialResponse`, `ExternalTaskResponse`, `PageVisit`	Réponses, essais, résultats de tâches externes, arrivée sur les pages
`Masquage`	Passages de réponses libres à masquer dans les exports
`Preregistration`, `PreregAmendment`, `PreregDeviation`	Préenregistrement, amendements, écarts
`NotebookEntry`	Entrées du cahier de laboratoire
`ActivityLog`, `Feedback`	Journal d'activité, retours des utilisateurs

Les réponses, essais, temps par page et masques sont rangés par identifiant d'étude et de participant, sans lien de base vers l'étude. Leur suppression passe par une fonction dédiée (`lib/donneesParticipants.js`), appelée quand une étude, un projet ou un compte est supprimé.

Exception volontaire : un préenregistrement verrouillé est conservé même si l'étude est supprimée, pour que ses liens restent vérifiables.

9.2 Types de questions (37)

Catégorie	Types
Choix unique	RADIO, SELECT, BUTTON_GROUP, MEDIA_RADIO, RADIO_COMMENT, DRILL_DOWN
Choix multiple	CHECKBOX, MEDIA_CHECKBOX, CHECKBOX_COMMENT
Texte libre	TEXT, INPUT_DEMAND, FILL_BLANK, WORD_LIST
Échelles	LIKERT, MATRIX, SLIDER, SEMANTIC_DIFF, SIDE_BY_SIDE, CONSTANT_SUM
Numérique	NUMERIC, EQUATION, COMPUTED
Affichage	DISPLAY, IMAGE, AUDIO, VIDEO
Spécial	RANKING, DATE, FILE_UPLOAD, TIMING, HOTSPOT, DRAG_DROP, DROP_WORD, HIGHLIGHT, RANDOM_CODE, META_INFO, CONSENT

10. Supervision

En clair. Le serveur tient un journal de son activité, et une adresse simple permet de vérifier qu'il répond.

- **Journaux applicatifs :** journaux structurés de Fastify (Pino).
- **Infrastructure :** outils de supervision de Scaleway.
- **État du serveur :** `GET /health` indique que le serveur répond (statut et heure). Cette route ne contrôle pas la base de données.
- **Actions des utilisateurs :** table `ActivityLog`.

> Les seuils d'alerte sont documentés dans la version intégrale.

11. Open source et communauté

En clair. Tout le monde peut lire le code et le réutiliser, à condition de republier ses modifications sous la même licence. Le service en ligne, lui, a ses propres conditions d'utilisation.

Le code est publié sur GitHub sous licence GNU AGPL-3.0-or-later. Cette licence couvre le code source. Le service hébergé sur www.mindcraft-research.fr reste régi par ses conditions d'utilisation, qui imposent un usage non commercial.

11.1 Identifiants pérennes

Type	Valeur
DOI Zenodo v0.2.0	10.5281/zenodo.23466120 (collectes à partir du 19 octobre 2026)
DOI Zenodo v0.1.0	10.5281/zenodo.19864887 (collectes antérieures)
SWHID v0.1.0	swh:1:rev:df8a8b127f4b7c024a140e56263d0e22927b33eb
ORCID de l'auteure	0000-0002-4315-1058
Dépôt	github.com/mindcraft-research/mindcraft

11.2 Fichiers de gouvernance

- `LICENSE`, `README.md`, `CONTRIBUTING.md`, `CODE_OF_CONDUCT.md`, `SECURITY.md`
- `CITATION.cff` et `codemeta.json` : métadonnées de citation
- `.github/dependabot.yml` : mises à jour automatiques
- `docs/json-export-format.md` : format d'export de la structure des études
- `docs/timing-validation/` : méthodologie, code et données de la validation temporelle

12. Moteur expérimental et mesures physiologiques

En clair. Les tâches s'exécutent dans le navigateur du participant, pour ne pas dépendre du réseau pendant les essais. Pour synchroniser un capteur (eye-tracker, EEG…), MindCraft envoie des repères datés à votre logiciel d'acquisition.

12.1 Moteur de tâches

Le moteur (`frontend/src/components/stimulus/`) s'exécute dans le navigateur. Le serveur intervient pour charger la configuration et enregistrer les réponses. Un essai enchaîne les étapes définies dans l'étude : intervalle, fixation, stimulus, masque, feedback.

12.2 Mesure du temps

Le temps de réaction est mesuré avec `performance.now()` , horloge du navigateur insensible aux ajustements de l'heure du système. Sa précision réelle dépend du navigateur, de l'ordinateur et de l'écran.

La méthode, le code et les données de la validation temporelle de MindCraft sont publics dans `docs/timing-validation/` .

> **Limite des études en ligne.** Pour un paradigme exigeant une précision de l'ordre de la milliseconde (masquage visuel, potentiels évoqués synchronisés au stimulus), préférez un environnement de laboratoire dédié.

12.3 Mesures physiologiques (LSL)

Lab Streaming Layer (LSL) est un protocole standard pour synchroniser plusieurs flux de données. MindCraft n'enregistre pas le signal : il envoie des marqueurs, que le logiciel d'acquisition (BrainVision, BIOPAC, Tobii Pro Lab, OpenBCI…) enregistre avec le signal.

Type de tâche	Marqueurs
Tous les blocs	Marqueurs globaux : `STUDY_START` , `BLOCK_START` , `PAGE_START` , `Q_SHOW` , `Q_RESP` …
Tâche interne	Automatiques, à chaque étape de l'essai : `F` , `S` , `R` , `FB` (personnalisables)
Tâche externe en iFrame	Envoyés par la tâche avec `postMessage` , et conservés côté serveur au fil de la tâche
Tâche externe en redirection	`TASK_START` et `TASK_END` uniquement

Le relais `docs/lsl-relay.py` , lancé sur l'ordinateur du participant, reçoit les marqueurs en WebSocket (port 12345 par défaut) et les publie dans le flux LSL « MindCraft-Markers ».

13. Contrebalancement et allocation

En clair. Pour que l'ordre de présentation n'influence pas les résultats, MindCraft fait varier cet ordre entre participants et répartit les personnes équitablement entre les conditions.

13.1 Méthodes

- **Carré latin :** chaque condition apparaît une fois à chaque position.
- **Séquence de Williams :** contrôle aussi l'effet d'une condition sur la suivante (formule de Williams, 1949, pour un nombre pair de conditions).
- **Aléatoire :** ordre tiré au hasard pour chaque participant.

13.2 Allocation

À l'arrivée d'un participant, MindCraft choisit la condition inter-sujets la moins remplie, et tire au sort en cas d'égalité. L'opération se déroule dans une transaction protégée par un verrou (section 8).

13.3 Quotas

- **Strict (par défaut) :** une condition pleine ne reçoit plus personne. Quand toutes le sont, l'étude est complète.
- **Souple :** les conditions les moins remplies restent prioritaires, avec une tolérance au-delà de l'objectif par condition.

Le quota compte les participants ayant commencé. Une passation marquée abandonnée libère sa place.

13.4 Reproductibilité

Le tirage utilise un générateur aléatoire sans graine fixée : il ne se rejoue donc pas à l'identique. En revanche, ce qui a été attribué est conservé pour chaque passation (condition, ordre des blocs) et figure dans les exports. Les analyses peuvent ainsi s'appuyer sur ce que chaque participant a réellement vu.

Annexe A. Routes de l'API

> **En clair.** La liste des adresses que le site utilise pour parler au serveur. Elle ne sert qu'aux développeurs et aux auditeurs.

Accès : **Public** (sans compte), **Compte** (connecté), **Lecture** (tout membre du projet), **Écriture** (propriétaire ou éditeur), **Propriétaire**, **Admin**. Pour les études, les préfixes sont `/api/studies`, sauf indication.

A.1 Authentification (`/api/auth`)

Routes	Accès	Rôle
`POST /register, /login, /2fa/login-verify, /refresh, /verify-email, /resend-verification, /forgot-password, /reset-password`	Public	Inscription et connexion
`GET /me, /data-export` ; `PATCH /profile, /onboarding/complete`	Compte	Profil et export des données personnelles
`POST /logout, /change-password, /2fa/setup, /2fa/verify, /2fa/disable, /delete-account`	Compte	Sécurité et suppression du compte

A.2 Projets (`/api/projects`)

Routes	Accès	Rôle
`GET /, /:id` ; `POST /` ; `PUT /:id` ; `DELETE /:id`	Selon le rôle	Gestion des projets
`POST /:id/invite` ; `DELETE /:id/collaborators/:collaboratorUserId`	Selon le rôle	Équipe
`GET /invitations/:token` ; `POST /invitations/:token/accept, /decline`	Lien d'invitation	Réponse à une invitation

A.3 Études, blocs, questions (`/api/studies`)

Routes	Rôle
`GET /, /:id` ; `POST /, /import, /:id/duplicate` ; `PATCH /:id, /:id/status` ; `DELETE /:id`	Études, import, statut
`GET, POST /:id/preview-key` ; `GET /:id/open-science/suggestions`	Lien d'aperçu, propositions Open Science
`GET /:id/sessions/count` ; `DELETE /:id/sessions, /:id/participants/:participantId`	Réinitialisation des données, effacement d'un participant
`POST /:id/blocks` ; `PUT, DELETE /:id/blocks/:blockId` ; `PUT /:id/blocks/reorder` ; `POST /:id/blocks/:blockId/duplicate, /copy-to-study`	Blocs

`POST /:id/blocks/:blockId/questions` ; `PUT /:id/blocks/:blockId/reorder-questions` ; `PUT, DELETE, POST .../questions/:questionId[/duplicate]`	Questions

A.4 Design et sessions (`/api/studies`)

Routes	Rôle
`GET, POST, PUT, DELETE /:id/design` ; `GET /:id/design/preview`	Design
`POST, PUT, DELETE /:id/design/factors[/:factorId[/levels[/:levelId]]]`	Facteurs et niveaux
`GET /:id/sessions, /:id/recruitment`	Suivi du recrutement
`POST /:id/sessions/allocate` ; `PATCH /:id/sessions/:sessionId`	Public : début et statut d'une passation

A.5 Passation (`/api/run`, public)

Routes	Rôle
`GET /:studyId`	Chargement de l'étude
`POST /:studyId/page-visit`	Arrivée sur une page
`POST /:studyId/responses/questions`	Réponses aux questions
`POST /:studyId/random-code`	Code aléatoire
`POST /:studyId/responses/external-task[/partial]`	Résultats d'une tâche externe

A.6 Tâches et fichiers

Routes	Rôle
`/api/stimulus/blocks/:blockId/upload, /text, /files, /sequence` ; `/api/stimulus/files/:fileId`	Fichiers et séquences des tâches
`POST /api/stimulus/responses`	Public : réponses aux essais
`GET /api/stimulus/studies/:studyId/responses`	Membres du projet : réponses de l'étude
`POST /api/media/upload` ; `GET /api/media/files/:filename`	Médias

A.7 Exports et anonymisation (`/api/studies`, écriture)

Routes	Rôle
`GET /:id/export/csv, /csv-trials, /csv-external, /csv-external-markers, /excel, /ods`	Données. `?brut=1` : réponses d'origine, propriétaire seulement
`GET /:id/export/codebook, /json, /structure-zip`	Documentation et structure
`GET /:id/freezes/:freezeId/codebook, /structure`	Version enregistrée au premier participant (lecture)

`GET /:id/anonymisation` ; `PUT /:id/anonymisation/:responseId` ; `POST /:id/anonymisation/partout, /valider`	Masquage des réponses libres

A.8 Préenregistrement et cahier

Routes	Rôle
`GET, PUT /api/studies/:id/prereg` ; `POST .../lock, /amendments` ; `PATCH .../public` ; `GET .../pdf`	Préenregistrement (verrouillage et lien public : propriétaire)
`/api/studies/:id/prereg/deviations[/detected, /:devId, /export]`	Écarts
`GET /api/prereg/:token, /:token/pdf`	Public : consultation par lien
`GET, POST /api/studies/:id/notebook` ; `POST .../depot` ; `GET .../pdf`	Cahier de laboratoire

A.9 Administration et divers

Routes	Rôle
`/api/admin/users, /stats, /prereg, /resend-verification, /reset-onboarding`	Admin
`/api/feedback`	Retours des utilisateurs
`GET /health`	Public : état du serveur

Annexe B. Variables d'environnement (résumé)

- **Serveur :** connexion à PostgreSQL, secret JWT, port, adresse de l'interface (CORS), clé du service d'e-mails. Voir `backend/.env.example`.
- **Interface :** adresse de l'API, intégrée au site à la construction. Les variables `NEXT_PUBLIC_` sont publiques et ne doivent jamais contenir de secret. Voir `frontend/.env.example`.
- **Intégration continue :** clés et identifiants Scaleway, conservés dans les secrets chiffrés de GitHub.

Annexe C. Glossaire

Terme	Définition
AGPL	Licence libre : qui modifie le code et le propose en service en ligne doit publier ses modifications.
API	Porte d'entrée du serveur : les adresses que le site utilise pour lire et écrire des données.
bcrypt	Méthode de hachage : transforme un mot de passe en une empreinte dont on ne peut pas revenir au mot de passe.
Bloc	Étape d'une étude : accueil, questionnaire, tâche, logique, message de fin.
Codebook	Document qui décrit chaque variable d'une étude.
Conteneur (Docker)	Paquet qui contient une application et tout ce qu'il lui faut pour fonctionner.
CORS	Règle qui indique quels sites ont le droit d'interroger le serveur.
CSRF	Attaque où un site malveillant déclenche une action à l'insu d'une personne connectée.
Dependabot	Robot de GitHub qui propose des mises à jour des dépendances.
DOI	Identifiant permanent pour citer une publication ou un logiciel.
DPO	Délégué·e à la protection des données.
HSTS	Consigne qui oblige le navigateur à utiliser HTTPS.
JWT	Jeton signé qui prouve qu'une personne est connectée.
LSL	Lab Streaming Layer : protocole de synchronisation d'enregistrements.
Marqueur	Repère daté indiquant un événement de l'étude dans un enregistrement.
Masque	Remplacement d'un passage de réponse libre par une étiquette dans les exports.
ORCID	Identifiant permanent d'un·e chercheur·e.
ORM (Prisma)	Outil qui fait le lien entre le code et la base de données.
Pseudonymisation	Remplacer l'identité par un identifiant. Un lien avec la personne reste possible, à la différence de l'anonymisation.
REST	Façon courante d'organiser les adresses d'une API.
SWHID	Identifiant d'archive attribué par Software Heritage.
TOTP	Code temporaire à six chiffres de la double authentification.
XSS	Attaque qui injecte du code malveillant dans une page web.